

Finding excited quantum states with neural networks: Physics-Informed Neural Networks with extra constraints

FYS-STK3155 - Project 3

Frederik Callin Østern, Heine Elias Husdal
Department of Physics, University of Oslo
(Dated: December 18, 2025)

We investigate using feed-forward neural networks to find the stationary solutions to the Schrödinger equation, as well as their energy eigenvalues. We employ a procedure similar to physics-informed neural networks, except that we use some extra constraints. In particular, we normalize the residual loss of the differential equation, and add a soft penalty term to the loss that ensures orthogonality between different states. To demonstrate the method on a simple problem, we use it to find the first few excited states of the quantum harmonic oscillator problem. For the first excited state, the total loss is optimized through a combination of a broad random search, and then a finer one, in order to find the best values of the relevant hyperparameters. This includes the number of nodes, hidden layers, learning rates, and relative weights between different loss terms. We end up with a wavefunction that closely matches the correct state, and with the correct energy eigenvalue up to an accuracy of 0.095 %. We also test the method on the second and third excited states. It is found that with just a sufficiently large orthogonality loss (and no optimization of the hyperparameters), it finds the correct energy up to an accuracy of 0.77 % and 2.39 %, respectively. However, by comparing the performance of neural networks with a simple finite difference approximation, we find that the latter consistently obtains a mean squared error several orders of magnitude smaller compared to the analytical solution. Despite that the finite difference method is more accurate here, the method seems to be sufficiently robust that it may be useful to find excited states for other, much less tractable problems.

I. INTRODUCTION

In physics, most laws can be formulated as ordinary or partial differential equations. Finding good numerical solvers for these equations is therefore indispensable to model systems in many different fields, such as quantum physics, climate modelling and engineering. Traditionally, there exist various finite difference and finite element methods that one can use to solve these equations on a grid [1].

However, these techniques may struggle on high-dimensional data, which could significantly increase computational cost. Ordinary data-driven machine learning techniques, on the other hand, do not encode any of the physics into the model, and they are not accurate if there is limited data [1]. To mitigate these concerns, one may use physics-informed neural networks (PINNs). These combine ideas from both methods, allowing the use of neural networks to encode the underlying laws and find smooth solutions to the differential equations on the desired domain. One may also add other constraints, such as boundary and initial conditions, or symmetry principles [2].

In this project, we have used some of the essential ideas of physics-informed neural networks to solve the one-dimensional Schrödinger equation for the harmonic oscillator problem. The work is a proof-of-concept to demonstrate the technique on a concrete problem. We used a PINN to find the ground state wavefunction and energy on a bounded interval, and then used it to solve for the first excited state. We added an extra soft orthogonality constraint to the cost function in order to get a

wavefunction orthogonal to the lower states. Moreover, we explored the addition of L2 regularization. For the first excited state, we optimized the hyperparameters of the model, including the learning rate, number of nodes, layers and activation function, as well as the weights on some of the penalty terms in the cost function. The optimization was done by a combination of a broad random search, and then a finer search on a smaller grid. The accuracy of our best results was compared to the results of an ordinary finite difference approximation. At the end, we also checked our results on higher excited states.

In Section II, we describe how to solve differential equations with neural networks, as well as the ordinary finite difference approximation. We also give an explanation of how our analysis was performed. In Section III, our results are presented and discussed. Finally, in Section IV, we end with a conclusion and possibilities for future work.

II. METHODS

A. Differential equations and neural networks

Our approach for solving differential equations (DEs) is loosely based on the method of using *Physics Informed Neural Networks* (PINNs) [2, 3]. The idea is to incorporate known physical laws and principles described by nonlinear differential equations into the training of a neural network. In particular, one trains the neural network to optimize a solution to a DE, together with boundary and/or initial conditions.

A general ordinary or partial differential equation (DE) may be written as [4]

$$f(\mathbf{x}, g(\mathbf{x}), \mathbf{D}^1 g(\mathbf{x}), \mathbf{D}^2 g(\mathbf{x}), \dots, \mathbf{D}^n g(\mathbf{x})) = 0 \quad (1)$$

Here, $\mathbf{x}$ is a vector of all independent variables, g is the scalar function that solves the differential equation, and $\mathbf{D}^p g$ denotes the set of all p th order partial derivatives of g . The differential equation is defined by the function f , and it has the order n . If one does not have a DE on the form of Eq. (1), then one may rearrange terms to achieve the desired form.

In order to solve the DE, the feed-forward neural network (FFNN) needs to find an appropriate function g that gives the best approximation to Eq. (1). That means one wishes to minimize the (squared) absolute value of f on a collection of selected input data points, called collocation points [2]. Thus, if one has a set $\{\mathbf{x}_i\}_{i=1}^M$ of data points, then an appropriate cost function is the mean squared value of f on these points [4],

$$C(\Theta) = \frac{1}{M} \sum_{i=1}^M (f(\mathbf{x}_i, g(\mathbf{x}_i, \Theta), \mathbf{D}^1 g(\mathbf{x}_i, \Theta), \dots, \mathbf{D}^n g(\mathbf{x}_i, \Theta)))^2 \quad (2)$$

Here, Θ denotes the collection of weights and biases in the neural network, and we have noted that the function g depends on these parameters.

In the input layer, the PINN takes in the collection of values in an input vector $\mathbf{x}$. The output layer contains a single neuron, corresponding to the value of the output scalar function. The neural network may then be constructed by a chosen set of hidden layers, neurons and activation functions, and it is trained by minimizing the cost function in Eq. (2) with the use of the backpropagation algorithm and gradient descent methods presented in our previous work [5, 6].

In order to get a unique solution to a differential equation, one must include boundary and/or initial conditions. There are two ways to implement this with neural networks. The method we will use is to write the final solution of the differential equation according to an ansatz [4]

$$g(\mathbf{x}, \Theta) = g_0(\mathbf{x}) + g_1(\mathbf{x})N(\mathbf{x}, \Theta) \quad (3)$$

where g_0 is a chosen trial function satisfying the boundary and/or initial conditions, g_1 is a function that is zero on the boundaries, and $N(\mathbf{x}, \Theta)$ is the output of the neural network on the input $\mathbf{x}$. With this ansatz, the solution to the differential equation will always satisfy

the initial and/or boundary conditions exactly.

The main method used in the literature on PINNs, is instead to add a penalty term to the cost function, corresponding to the error between the value of the neural network on the boundaries, and the desired boundary conditions [2, 3]. In particular, the penalty is the mean squared error between the desired and predicted values on the boundaries. With such an extra term, this error will also tend to be minimized as the feed-forward neural network is trained. However, considering that Eq. (3) gives an exact match with the initial and/or boundary conditions, we decided that it was the better choice.

B. Solving Schrödinger's equation with neural networks

One can write the one-dimensional time-independent Schrödinger equation in the form of Eq. (1) as [7]

$$f(x, \psi, \frac{d^2 \psi}{dx^2}) = -\frac{1}{2} \frac{d^2 \psi}{dx^2}(x) + V(x)\psi(x) - E\psi(x) = 0, \quad (4)$$

where ψ is the wavefunction that we wish to solve for, E is the energy eigenvalue, and V is the potential energy. Here, we have chosen units with mass $m = 1$ and reduced Planck constant $\hbar = 1$ for simplicity.

The goal of this work is to implement PINNs to recover the first few excited states, as well as energy eigenvalues, of a quantum system. Concretely, we focus on the quantum harmonic oscillator problem, which is given by the potential energy [7]

$$V(x) = \frac{1}{2}x^2, \quad (5)$$

where we choose units with angular frequency $\omega = 1$.

For reference, we may note that the normalized analytical solutions to the Schrödinger equation for harmonic oscillator problem, in our units, can be written as [7]

$$E_n = n + \frac{1}{2} \quad (6)$$

$$\psi_n(x) = \frac{1}{\sqrt{2^n n! \pi^{\frac{1}{2}}}} e^{-\frac{x^2}{2}} H_n(x)$$

where H_n is the n -th Hermite polynomial, E_n is the n -th energy eigenvalue and ψ_n the n -th eigenstate.

Since this is a one-dimensional problem, the input layer of the neural network will have a single neuron. In order to use the neural network to find the energy eigenvalues, we will include E as an additional parameter in the model that will be trained together with the weights and biases.

This approach is inspired by the data-driven method to discover the parameters of nonlinear partial differential equations [8]. Concretely, one may imagine that the differential equation is implemented as an extra last layer in the network, where each parameterized term in the expression for f in Eq. (1) corresponds to a neuron within this layer (and the non-parameterized term(s) can be included as an extra bias term). Then the extra parameter can be thought of as a weight in this layer.

Ansatz

For the harmonic oscillator problem, the boundary condition is that the wavefunction must vanish at infinity,

$$\psi(x) \rightarrow 0 \quad \text{when} \quad |x| \rightarrow \infty. \quad (7)$$

In order to choose an appropriate ansatz, however, we may also use some of our knowledge of the physics. In particular, in order to get a normalized solution of the harmonic oscillator problem, the wavefunction must drop exponentially by a factor of $e^{-\frac{x^2}{2}}$ (in our units) [7]. Thus, an appropriate ansatz of the form of Eq. (3) satisfying the boundary conditions is

$$\psi(x, \Theta) = e^{-\frac{x^2}{2}} N(x, \Theta) \quad (8)$$

By including the appropriate asymptotic behavior in the ansatz, one may stabilize the training. Importantly, we use this same ansatz both for the ground state and for the excited states. Any unique structure in the excited states must therefore arise solely from the physics constraints and not from the ansatz.

Training and cost function

In order to solve the Schrödinger equation, there are two extra considerations to be made. First, the resulting wavefunction must be normalized, so that the total probability adds to 1. Second, since there are many energy eigenstates, one must make sure the solution converges to the desired state.

One way to solve the problem with normalizability, is to add an extra penalty to the cost function that is proportional to the squared difference between the integral of the squared wavefunction, and the value 1. The integral may be estimated by Monte Carlo integration, using the wavefunction at the collocation points. However, the problem with this approach is that without an optimization of the initial weights and biases of the network, it may take many iterations of the gradient descent algorithm before the network ends up with an approximately normalized wavefunction.

Moreover, one would get extra computational cost due to the need of optimizing the relative weight of the original cost in Eq. (2) and the extra normalizability penalty term. This may be difficult, since the cost in Eq. (2) would be minimized for vanishing wavefunctions, leading to a "tug of war" between the two cost terms and hence possible unstable convergence.

Instead, we decided to modify the cost function by using the normalized differential equation residual loss

$$C_{DE}(E, \Theta) = \frac{\sum_{i=1}^M (-\frac{1}{2} \frac{d^2 \psi}{dx^2}(x_i, \Theta) + V(x_i) \psi(x_i, \Theta) - E \psi(x_i, \Theta))^2}{\sum_{i=1}^M \psi(x_i, \Theta)^2} \quad (9)$$

By dividing by the sum of squared wavefunction values (which represents the total probability), we have *effectively* normalized the wavefunction in the cost function. This is done without imposing a hard normalization constraint on the output of the wavefunction, which could also lead to instability.

In order to find the ground state, we start with a sufficiently small initial value of the energy. Then one may expect that the energy will converge to the ground state energy. However, one cannot expect to find the higher excited states directly. The reason is both that we cannot assume that we know the excited energy levels (so that we do not know what would be an appropriate initial energy), but also because any convergence to an excited state may be unstable - instead of converging to these states, the energy may end up dropping until it settles on the ground state again. Thus, to find the n -th state, we must train the neural network while effectively *excluding* the $n-1$ lower states.

To do this, we may again exploit our knowledge of the physics of the problem, namely that different energy eigenstates are orthogonal. Thus, to exclude the $n-1$ lower states and find the n -th state, we may add an extra penalty

$$C_{ortho}^n(\Theta) = \sum_{i < n} \left(\frac{\sum_{j=1}^M \psi^i(x_j) \psi(x_j, \Theta)}{\sqrt{\sum_{k=1}^M \psi^i(x_k)^2 \sum_{m=1}^M \psi(x_m)^2}} \right)^2 \quad (10)$$

Here, ψ^i is the i -th energy eigenstate that has previously been found with neural networks, and ψ is the current eigenstate that we wish to find. This penalty is proportional to the inner product between the states, and we have again divided by the squared norm of the two states in order to normalize the penalty. With this normalization, the penalty will not be sensitive to the scaling of the two states, but only on their mutual overlap, which will be a number bounded between the

values -1 and 1 .

The total cost function for the n -th eigenstate may then be written as

$$C^n(E, \Theta) = C_{DE}(E, \Theta) + \lambda_{ortho} C_{ortho}^n(\Theta) + \lambda_{decay} \sum_{\theta \in \Theta} |\theta|^2 \quad (11)$$

Here, we have introduced a parameter λ_{ortho} to ensure the proper relative weight between C_{DE} and C_{ortho} . Moreover, in general, we may include a L2 regularization term, proportional to the sum of squared values of all the weights and biases $\theta \in \Theta$, with a relative weight of λ_{decay} . Both λ_{ortho} and λ_{decay} are extra hyperparameters that must be optimized.

To summarize, we end up with a precise prescription to iteratively find the eigenstates and energy eigenvalues with neural networks. First, we find the ground state by choosing an initial value for the energy parameter that is sufficiently small. This can be chosen by utilizing knowledge about its general properties (such as that the ground state energy must be nonnegative, which means that one can choose a small positive value). Then, to find the n -th excited eigenstate (where $n \geq 1$), one can use the cost in Eq. (11) to train the network, with the initial energy being equal to the energy of the $n-1$ -th eigenstate (where the 0-th eigenstate corresponds to the ground state) that one has previously found with neural networks. This is the method we used in this work.

C. Numerical Finite Difference Solution

The results of the neural network analysis may be compared to a simple finite difference approximation of the solution to the Schrödinger equation. We choose an appropriately large interval $[-L, L]$ for some $L > 0$, and discretize the points in this interval according to $x_i = -L + i \cdot \Delta x$ for $0 \leq i \leq n+1$, where $\Delta x = \frac{2L}{n+1}$ and $n+2$ is the number of points. One may then approximate the second derivative with respect to x according to the centered difference

$$\frac{d^2\psi}{dx^2}(x) \rightarrow \frac{\psi(x + \Delta x) - 2\psi(x) + \psi(x - \Delta x)}{\Delta x^2} \quad (12)$$

If L is sufficiently large, then we may approximate the wavefunction as being equal to zero at $x = -L$ and $x = L$. Using these boundary conditions, and plugging Eq. (12) into Eq. (4), we get

$$A\psi = E\psi \quad (13)$$

where ψ is a n -dimensional vector whose i -th component is equal to $\psi(x_i) = \psi(-L + i \frac{2L}{n+1})$, and A is a $n \times n$ tridiagonal matrix with diagonal elements

$$A_{i,i} = \frac{1}{\Delta x^2} + V(x_i) \quad (14)$$

and off-diagonal elements

$$\begin{aligned} A_{i,i-1} &= -\frac{1}{2\Delta x^2} \quad \text{for } i > 1 \\ A_{i,i+1} &= -\frac{1}{2\Delta x^2} \quad \text{for } i < n \end{aligned} \quad (15)$$

From Eq. (13), we see that for each eigenstate n , we may find the numerical solution to the Schrödinger equation by finding the eigenvector of A whose eigenvalue is closest to the analytical energy eigenvalue in Eq. (6).

D. Implementation

1. Training pipeline

All models were implemented in **PyTorch** and trained using automatic differentiation to evaluate the spatial derivatives required by the Schrödinger equation. In each training step we computed $\psi(x, \Theta)$ and obtained $\frac{d^2\psi}{dx^2}$ by differentiating with respect to the input tensor x using **torch.autograd** twice.

For the harmonic oscillator we sampled 256 training points uniformly on a finite interval $x \in [x_{\min}, x_{\max}] = [-5, 5]$. The training loop was run for a fixed number of 3000 epochs for all of the analysis in this project.

We used the Adam gradient descent method to train the neural networks and optimize the hyperparameters. This is done with the **torch.optim.Adam** method from PyTorch. Since the eigenvalue E is treated as a trainable scalar parameter, it is optimized together with the network weights and biases using a separate learning rate η_E . L2 regularization is included through the optimizer's **weight_decay** argument, corresponding to the λ_{decay} term in Eq. (11).

To reduce the sensitivity to fluctuations in late epochs during training, as well as potential nonconvergence, we make a checkpoint of the model whenever the loss improves relative to the previous minimal loss. After the final epoch we restore the best-loss checkpoint and use this model for evaluation and plotting. Thus, throughout all of the analysis shown in this project, the final neural network we end up with always corresponds to the epoch with the smallest loss value among all of the 3000 epochs.

2. Hyperparameter Search

The hyperparameters were empirically tuned using a combination of random search and targeted grid searches. We first fixed a baseline configuration shown in Table I

and performed a broad random search over the learning rate η , the energy learning rate η_E , and the orthogonality weight λ_{ortho} . The random search was orchestrated with the *Weights & Biases* (W&B) sweep service [9], and we launched approximately 1000 runs.

Table I: Baseline configuration used as a starting point for the hyperparameter searches.

Quantity	Value
Hidden layers	3
Neurons per hidden layer	64
Hidden layer activation function	tanh
Weight decay parameter λ_{decay}	0

During training, the gradient descent algorithm minimizes the weighted objective in Eq. (11). To compare runs in a consistent way, we evaluated models on an independently generated set of test points x_{test} drawn from the same uniform distribution on $[-5, 5]$. We logged the differential equation (DE) residual C_{DE} (Eq. (9)) and the orthogonality penalty C_{ortho}^n (Eq. (10)), and their sum $C_{sum} = C_{DE} + C_{ortho}^n$ as a metric to compare different model configurations. This evaluation metric is not theoretically optimal. In particular, C_{DE} and C_{ortho}^n measure different aspects of the solution and are not guaranteed to be directly comparable in scale. However, it provides a practical single-number indicator that worked well to identify configurations where both costs are simultaneously low.

After the broad search, the finer grid search was performed in a smaller range of values for some selected pairs of hyperparameters. These ranges were chosen based on the values with the best performance during the broad search. Moreover, to get more robust neural network predictions, we used a procedure that is similar to k-fold cross-validation [5]. In particular, for each configuration of hyperparameters in the finer grid search, we did a number $n_{repeats}$ of independent runs, each with randomly generated input data, in order to train the network. At the end, we averaged the metrics used (in particular, the various loss terms). With this procedure, one reduces model bias caused by the specific training data that is used.

E. Use of AI tools

In this project, we have utilized Large Language Models (LLMs) such as ChatGPT to assist with various aspects of the project. In particular, it was helpful in the following areas:

- Becoming familiar with how to use the library `pickle` to save Python objects.
- Finding some relevant articles about physics-informed neural networks, in particular, [1] and [3].

III. RESULTS AND DISCUSSION

A. Ground state and initial excited state attempt

As an initial check, we trained a model for the ground state of the harmonic oscillator using a set of reasonable hyperparameters without any systematic tuning. The configuration is summarized in Table II. With this setup, the model converged to the exact ground state energy $E_0 = 0.5$ predicted by Eq. (6), and the learned wavefunction closely matched the analytical solution.

A key reason why the ground state is comparatively easy to learn in our framework is the choice of ansatz in Eq. (8). For the harmonic oscillator, the ground state wavefunction is just proportional to $e^{-\frac{x^2}{2}}$ [7]. Since this factor is already built into the ansatz, the neural network only needs to make sure the trainable parameter E converges to the correct eigenvalue.

Table II: Guessed hyperparameter configuration used to train the ground state.

Quantity	Value
Hidden layers	3
Neurons per hidden layer	32
Hidden layer activation function	tanh
Learning rate η	$5 \cdot 10^{-2}$
Energy learning rate η_E	10^{-1}
Initial energy E_{init}	0.1
Weight decay parameter λ_{decay}	0

We then attempted to learn the first excited state using the same ansatz as for the ground state (Eq. (8)), and enforced orthogonality to the learned ground state through the penalty term C_{ortho}^1 in Eq. (10). In this small test, we again used the same hyperparameters as in Table II, and added an orthogonality penalty with weight $\lambda_{ortho} = 1000$. Smaller values of λ_{ortho} tended to collapse back to the ground state for this configuration. The resulting wavefunction, shown in Figure 9a, captures some of the structure of the first excited state, including a single node and approximate odd symmetry, despite no odd-parity factor being built into the ansatz.

However, the energy eigenvalue did not converge to the correct value. The predicted energy was $E \approx 1.10$, corresponding to a relative error of approximately 26.6% compared to the exact value of $E_1 = 1.5$ given by Eq. (6). This motivated a systematic hyperparameter search to improve the convergence for the excited state.

B. Hyperparameter search

The initial attempt in the previous subsection demonstrated that enforcing orthogonality is necessary to avoid collapse to the ground state, and that the excited state training is sensitive to the choice of hyperparameters.

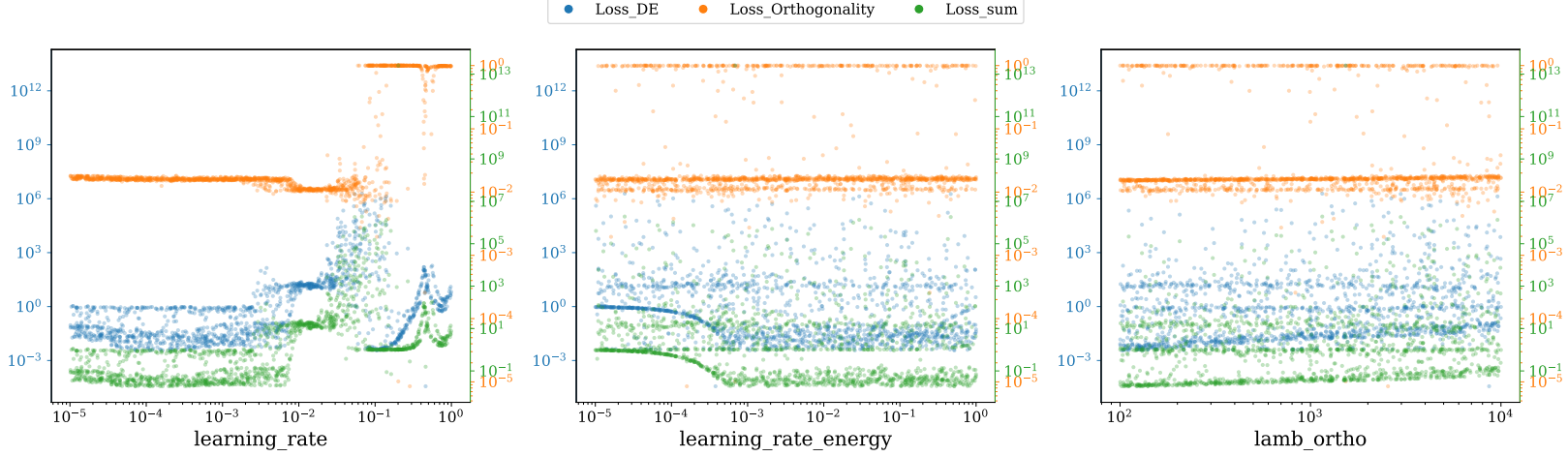


Figure 1: **Broad random search.** Cost/loss of neural network as a function of the ordinary learning rate η (the figure on the left), the learning rate for the energy parameter η_E (the figure in the middle), and the orthogonality loss weight λ_{ortho} from Eq. (11) (the figure on the right). We show both the residual loss C_{DE} , the orthogonality loss C_{ortho}^1 and the total loss $C_{sum} = C_{DE} + C_{ortho}^1$.

As described in the hyperparameter search procedure in Section IID2, we tuned the hyperparameters by ranking different configurations using the $C_{sum} = C_{DE} + C_{ortho}^1$ metric computed on an independent set of test points. We first performed a broad random search followed by targeted sweeps and grid searches.

We started from the baseline configuration in Table I and ran a broad random search over the ordinary learning rate η , the energy learning rate η_E , and the orthogonality weight λ_{ortho} . The searched ranges were $\eta \in [10^{-5}, 10^0]$, $\eta_E \in [10^{-5}, 10^0]$, and $\lambda_{ortho} \in [10^2, 10^4]$, with all parameters sampled log-uniformly. The initial λ_{ortho} range was motivated by the pre-search experiment in the previous subsection, where a value in the order of 10^3 was required to prevent the model from converging back to the ground state. Moreover, we used an initial energy of 0.5 in all of the hyperparameter search, since it is the energy we found for the ground state.

The results of the broad random search are summarized in Figure 1. While the dependence on λ_{ortho} is not immediately clear when viewing all runs, the plots for η and η_E show distinct regions where the total metric C_{sum} becomes small. In particular, it is clear that the lowest-loss runs concentrate at relatively small η and relatively large η_E .

To visualize this more directly, we plotted the random search runs in the (η, η_E) plane with C_{sum} encoded by color, shown in Figure 2. This plot reveals a very distinct rectangular region with low loss. In order to analyze λ_{ortho} further, we therefore restricted attention to runs with $\eta \leq 4 \cdot 10^{-3}$ and $\eta_E \geq 6 \cdot 10^{-4}$, which is within this region.

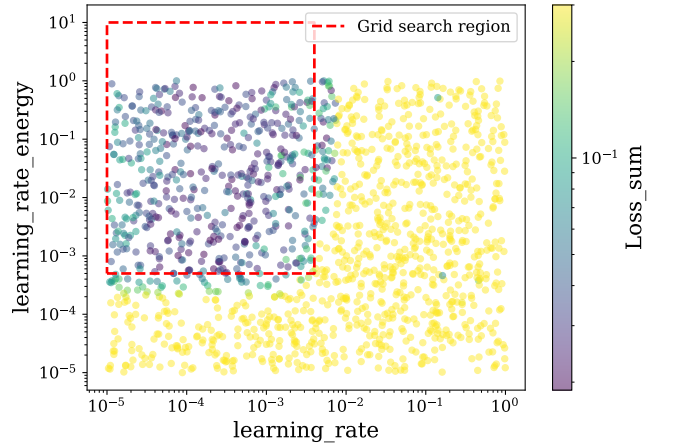


Figure 2: Scatter plot showing the performance (in terms of the total loss) as a function of the ordinary learning rate η , and the energy learning rate η_E , from Figure 1. Here, we can more clearly make out the range of values for these parameters where the total loss is smallest, and hence where we will perform the finer grid search.

In Figure 3, we show the loss terms as a function of λ_{ortho} for the corresponding filtered set of runs. From this figure, the dependence on the value of the orthogonality weight becomes more clear. We see that smaller values of λ_{ortho} tend to give smaller C_{sum} , with no indication that the optimum lies within the high range of λ_{ortho} we originally chose. This suggests that the orthogonality penalty needed to be much weaker than initially assumed, provided that η and η_E are chosen in this stable range.

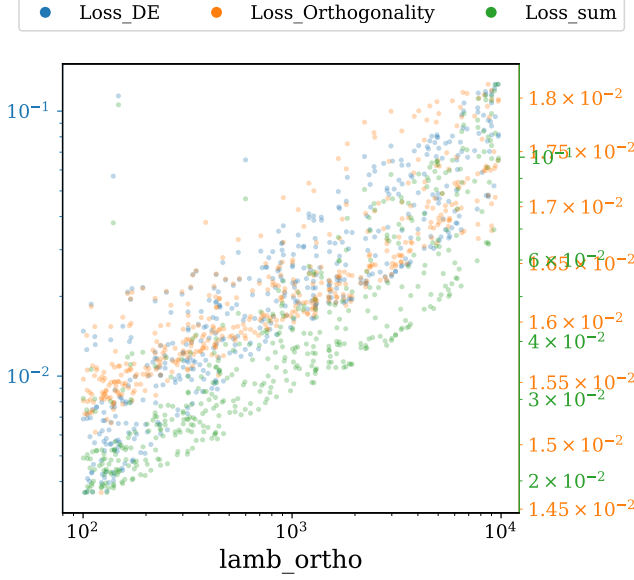


Figure 3: Cost/loss of neural network (both the residual loss C_{DE} as *Loss_DE*, C_{ortho}^1 as *Loss_Orthogonality* and *Loss_sum* as the sum of the two) as a function of orthogonality loss weight λ_{ortho} for learning rates $\eta \leq 4 \cdot 10^{-3}$ and energy learning rates $\eta_E \geq 6 \cdot 10^{-4}$. These ranges are within the grid search window in Figure 2. The trend in the plot shows that even lower λ_{ortho} may give better results.

Motivated by this trend, we performed a dedicated search for λ_{ortho} at lower values. We fixed $\eta = 10^{-4}$ and $\eta_E = 10^{-2}$, both within the favorable region identified above, and evaluated 40 log-spaced λ_{ortho} values in the interval $[10^{-1}, 10^2]$. For each λ_{ortho} , we averaged C_{sum} over $n_{repeats} = 5$ runs. The resulting curve is shown in Figure 4, and it shows a clear and sudden drop in C_{sum} . For $\lambda_{ortho} < 1$, the metric remains close to $C_{sum} \approx 10^0$, while at $\lambda_{ortho} \approx 1$ the loss drops sharply by several orders of magnitude. After that, it slowly increases again for larger λ_{ortho} . A natural interpretation is that for a too small λ_{ortho} , the orthogonality constraint becomes too weak and the model collapses to the ground state. If the state ends up being roughly the same as the ground state, then the orthogonality loss in Eq. (10) will be approximately equal to 1, which explains why C_{sum} ends up with approximately the same value. Increasing λ_{ortho} beyond this critical value forces the model away from the ground state and into an excited-state solution, producing a sharp improvement. Based on this transition, we chose a value $\lambda_{ortho} = 1.10$ slightly larger than the threshold to be sure the neural network would converge to the correct state, while also avoiding an overly strong orthogonality constraint.

With λ_{ortho} fixed at 1.10, we performed a finer grid search over η and η_E . The search region is within the range of values at which C_{sum} was small, and is shown as

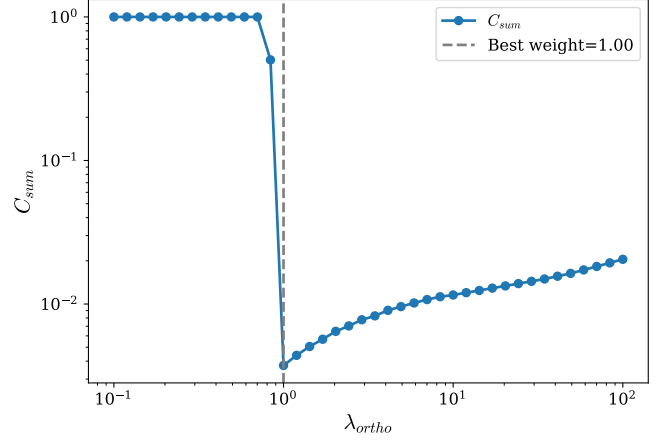


Figure 4: Total loss C_{sum} as a function of orthogonality loss weight λ_{ortho} for a smaller range of values of λ_{ortho} than in Figure 3. The learning rate $\eta = 10^{-4}$ and the energy learning rate $\eta_E = 10^{-2}$ is used. We find that the loss drops sharply at the specific value of $\lambda_{ortho} = 1.00$, before it begins increasing slowly again.

the red dotted region in Figure 2. It extended to slightly higher values of η_E than the upper bound for the original runs, since the random search did not reveal a clear lower bound in C_{sum} . Concretely, we searched in the ranges $\eta \in [10^{-5}, 4 \cdot 10^{-3}]$ and $\eta_E \in [5 \cdot 10^{-4}, 10^1]$ on a 10×10 grid, and averaged C_{sum} over $n_{repeats} = 4$ independent runs for each pair. The resulting heatmap is shown in Figure 5. The best performing combination was $\eta = 7.37 \cdot 10^{-5}$ and $\eta_E = 4.08 \cdot 10^{-2}$, with an average value of $C_{sum} = 3.99 \cdot 10^{-3}$. The fact that the learning rate for the energy is substantially larger than the ordinary learning rate may be understood from the fact that the energy must increase by a relatively large amount, meaning that a large learning rate is required to lift it to the necessary value.

Next, we optimized the network architecture by performing a grid search over the number of hidden layers and number of neurons per hidden layer. We tested 1 to 6 hidden layers and a number of neurons equal to all integer powers of 2 from $2^3 = 8$ to $2^9 = 512$. This time, we averaged over $n_{repeats} = 30$ runs, since each pair of values did not take too long to run. The resulting heatmap is shown in Figure 6. The best architecture in this search had 4 hidden layers with 256 neurons per hidden layer, producing an average of $C_{sum} = 2.61 \cdot 10^{-3}$. This indicates that, although the ground state can be learned with a relatively small network, the excited state problem benefits from a substantially larger network.

After selecting the learning rates, orthogonality weight, and architecture, we compared different activation functions in the hidden layers. Figure 7 shows the average C_{sum} for tanh, ReLU and sigmoid, computed for $n_{repeats} = 30$ independent runs per activation function.

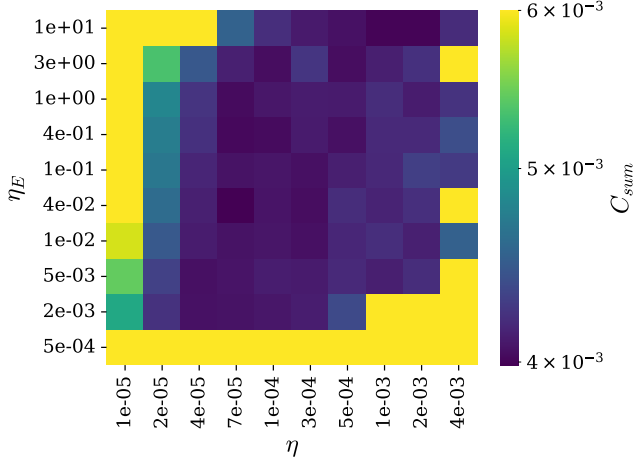


Figure 5: *Finer grid search for the learning rate η and the energy learning rate η_E . The total cost C_{sum} is shown as a function of these two parameters.*

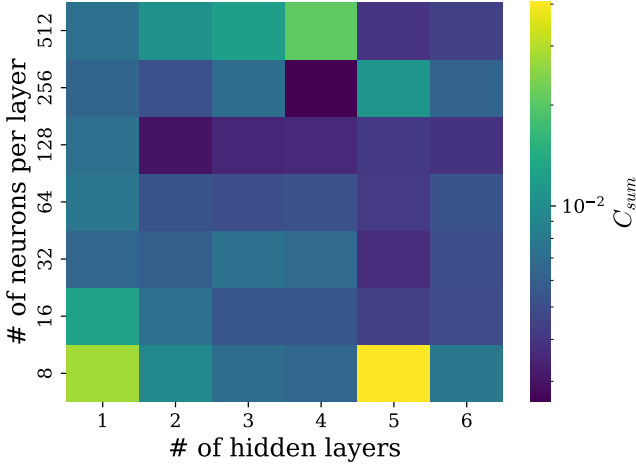


Figure 6: *Grid search for the neural network architecture, given as the number of nodes in each layer and the number of hidden layers. The total cost C_{sum} is shown as a function of these two parameters.*

The tanh activation performed best. This might be because the other parameters were chosen by training a network already consisting of tanh, as this was the activation function in our baseline configuration in Table I. The Sigmoid function resulted in $C_{sum} \approx 1$, suggesting that the model frequently collapsed to the ground state for the 30 runs.

The fact that the tanh function performed best, could also indicate that the neural network did not suffer from the vanishing gradient problem. The tanh function may risk suffering from this issue, since it is similar to the Sigmoid function in that its values saturate for very small or large inputs. However, the fact that it performed better than both ReLU and Sigmoid which, unlike the tanh function, both have the issue that their

average value is not equal to zero, indicates that the network preferred an activation function that did not suffer from introducing such a bias.

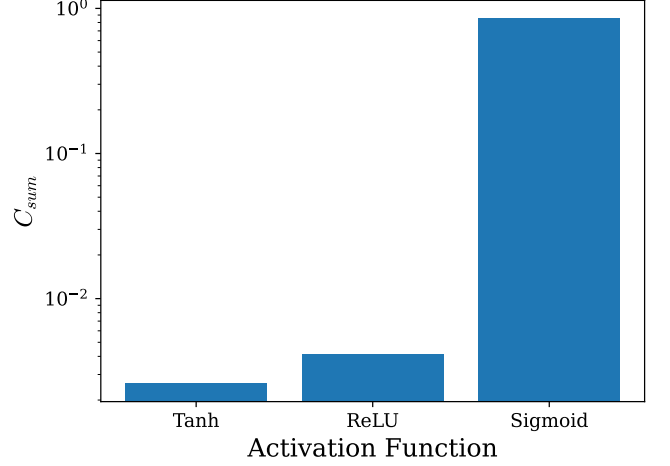


Figure 7: *The total cost C_{sum} for different choices of activation functions in the hidden layers. The best values of the learning rate η , learning rate for the energy η_E , the number of nodes and hidden layers and the orthogonality loss weight λ_{ortho} from the previous analysis have been used.*

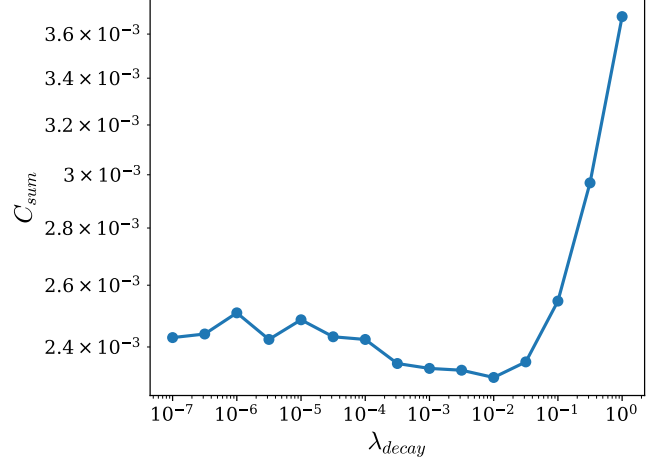


Figure 8: *The total cost C_{sum} as a function of the weight decay factor λ_{decay} from Eq. (11). The best values of all other hyperparameters from the previous analysis, including the activation function, have been used.*

Finally, we tested the effect of L2 regularization through the weight decay parameter λ_{decay} in Eq. (11). We evaluated 15 log-spaced values between 10^{-7} and 10^0 , averaging over $n_{repeats} = 10$ independent runs. The resulting curve in Figure 8 shows a clear minimum around $\lambda_{decay} = 10^{-2}$, which we therefore selected for

the final model. It led to a final loss of $C_{sum} = 2.31 \cdot 10^{-3}$.

The value of λ_{decay} is not very large, but the fact that the best value was larger than zero indicates that, unlike in our previous work [5, 6], the model does prefer a slight reduction of the effect of some of its many features. However, we see in the figure that the reduction should not be too large, as C_{sum} rises sharply for larger values of λ_{decay} . Thus, it is important for the model to keep a large number of features.

Combining the best values from these searches, our final configuration is summarized in Table III.

Table III: Final configuration and hyperparameters of the first excited state model, after a systematic search for the optimal parameters.

Quantity	Value
Orthogonality loss weight λ_{ortho}	1.10
Learning rate η	$7.37 \cdot 10^{-5}$
Energy learning rate η_E	$4.08 \cdot 10^{-2}$
Hidden layers	4
Neurons per hidden layer	256
Hidden layer activation function	tanh
Weight decay parameter λ_{decay}	10^{-2}

The final model with this configuration produced a predicted energy $E = 1.50143$. Compared to the exact value $E_1 = 1.5$, this has a relative error of 0.0951%. The final, improved wavefunction is illustrated in Figure 9b. We see that the optimized model closely matches the analytical solution of the first excited state.

C. Comparison to the numerical solution

In Figure 10, we compare the neural network results with the results of the numerical finite difference method (given by the eigenvector solution to Eq. (13) with the analytical eigenvalue for $n = 1$ from Eq. (6)) for the first excited state of the quantum harmonic oscillator. For each number of data points n , we train the neural network by picking n random collocation points in the interval $[-5, 5]$. For the finite difference method, we find the values of the wavefunction on a grid of data points in the interval $[-5, 5]$, starting at the value -5 and ending at the value 5 , with a constant step size between consecutive points. We show the mean squared error between the neural network or numerical wavefunctions and the analytical solution in Eq. (6) as a function of the number of data points at which the wavefunction values are computed. The data points at which it is computed are the same grid as the one used to find the finite difference approximation.

We see that for this problem, the numerical method

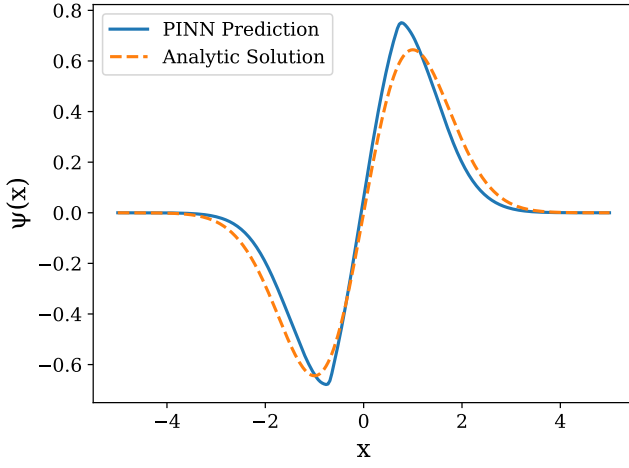
performs far better. Even for 10 data points, it has a mean squared error of 10^{-2} , which is on the order of 10 times smaller than the mean squared error of the neural network. Thus, it appears that the simple finite difference scheme would be better on this problem.

We notice that for sufficiently large numbers of data points, the mean squared error of the numerical scheme decreases by about 2 orders of magnitude when the number of data points increases by a factor of 10. Thus, it has a rate of convergence of 2, which may be expected for the centered difference approximation of second-order derivatives. However, it is seen that the mean squared error of the neural network is roughly constant as a function of the number of data points. Although it did not occur here, this suggests that for more complicated quantum mechanical problems, there may exist a sweet spot where the number of data points is small enough that neural networks would perform better than finite difference approximations. This illustrates the point that neural networks may perform better with limited data.

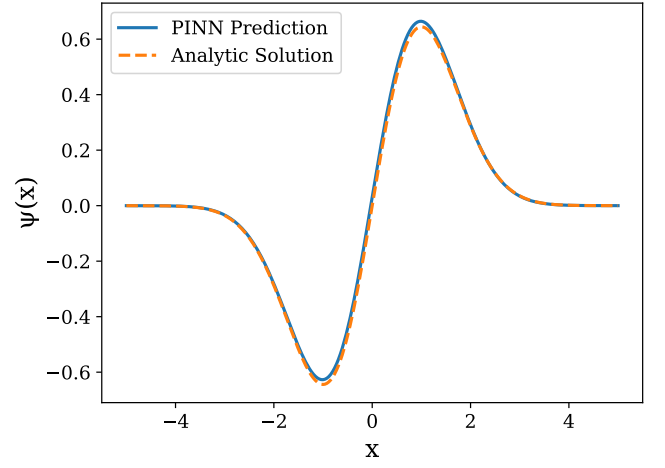
D. Other higher excited states

We end by applying our method to find the second and third excited states as well. Hence, we use the ground state ψ^0 and first excited state ψ^1 found from the previous analysis in Eq. (10) in order to define the orthogonality loss of the second excited state. As initial energy, we use the energy of 1.5 for the first excited state (note that in general, one would use the energy that one found from the neural network analysis, not the analytical value (as that is the value we are trying to find!)). In this case, the energy is sufficiently close to 1.5 that we are justified in using it). To find the third excited state, we may then use the first three states found from the previous analyses, as well as the initial energy 2.5 (again, for a similar reason as above). We do not optimize the hyperparameters this time, since we are only interested in obtaining a qualitative check that our method does generalize. Thus, we use the same hyperparameters as the optimal ones for the first excited state. The only difference is that we use a larger value of λ_{ortho} to ensure orthogonality; we use a value of 4.0 for the second excited state and a value of 15.0 for the third excited state.

The resulting comparison between the second and third neural network excited states, and the analytical wavefunctions, is shown in Figure 11. We see that the neural network does, indeed, give wavefunctions that are close to the analytical ones. This shows that our method is capable of identifying fairly accurate states, even for higher excited states and with no effort to optimize the hyperparameters. It was found that for the second excited state, the energy was within 0.77 % of the analytical value, and for the third excited state, the percentage was



(a) Before tuning hyperparameters.



(b) After tuning hyperparameters.

Figure 9: Comparison of the neural network (PINN) prediction with the analytical first excited state before and after hyperparameter tuning.

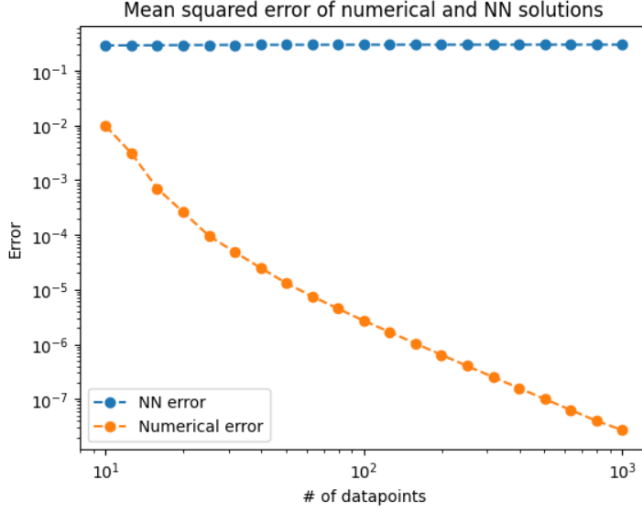


Figure 10: Mean squared error between neural network (NN) or numerical solutions and the analytical solution, as a function of the number of data points.

2.39 %. These errors are also reasonably small (though significantly larger than the percentage we got from the optimal hyperparameters of the first excited state).

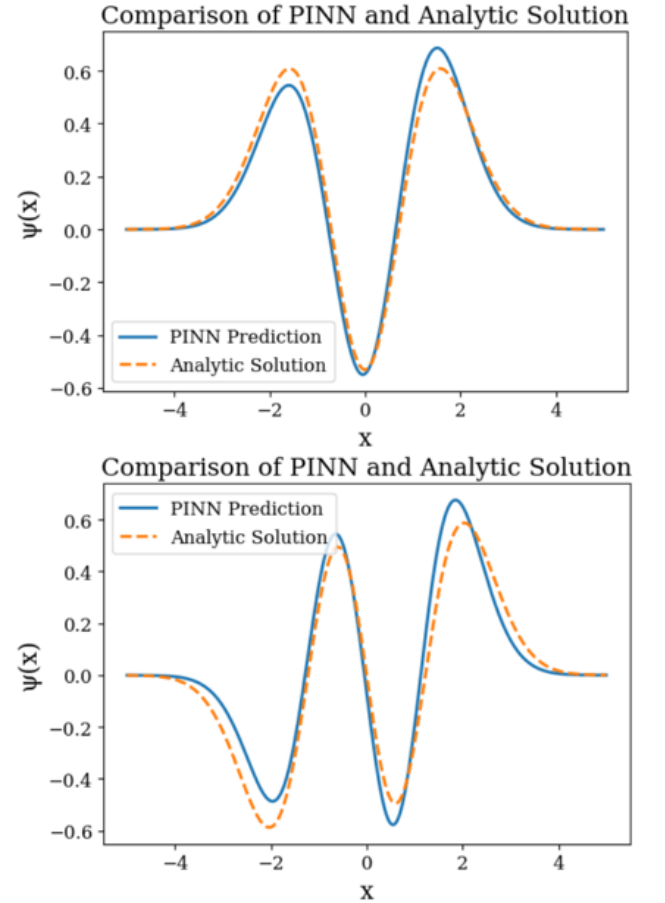


Figure 11: Comparison between the PINN prediction for the excited state of the quantum harmonic oscillator, and the analytical result. Top: The second excited state. Bottom: The third excited state.

IV. CONCLUSION

In this project, we implemented feed-forward neural networks with PyTorch to solve the one-dimensional Schrödinger equation for the quantum harmonic oscillator problem. We modified the loss of the differential equation to take the normalization of the wavefunction into account, and we added an extra orthogonality loss to ensure that the neural network would exclude all lower energy eigenstates. Furthermore, we used a special ansatz of the wavefunction to enforce the correct boundary conditions. We found that with no optimization of the hyperparameters, the neural network was capable of finding a relatively accurate solution for both the first, second and third excited states. For the first excited state, the accuracy of the energy improved from about 26.6 % to 0.095 % after the hyperparameters had been optimized. With the same hyperparameters, but only a slightly larger weight of the orthogonality term in the cost function, the energy ended up being within 0.77 % and 2.39 % of the correct value for the second and third state, respectively. With that said, the finite difference approximation performed much better, with a mean squared error about 10 times smaller than the error of the neural network for 10 data points, and with about 2 orders of magnitude bigger difference in error for each 10-fold increase in the number of data points.

We found that there was a sudden shift in the quantum state that the neural network converged to at a very

specific value of the weight of the orthogonality loss. Thus, the neural network rarely ended up "in between" two quantum states, which would lead to a problematic superposition of different states, rather than a single excited state. This shift suggests that the method of adding such a loss term works remarkably well, and that it may be used in future studies to identify excited states of more complicated problems.

We also identified that the neural network had almost the same mean squared error for a wide range of different numbers of data points that it was trained on. Since the mean squared error of the finite difference approximation decreases rather quickly with the number of data points, it indicates that there may be situations where neural networks end up with a much smaller error.

It is clear that our use of neural networks was rather limited by the simplicity of the problem we analyzed. To fully appreciate the advantage of machine learning methods, a future study could therefore either try a many-body system, which would lead to a high-dimensional problem, or it could analyze problems with other, highly nonlinear potential energies. One could have also used a more thorough multi-objective machine learning method to minimize the residual loss and orthogonality loss together, rather than minimizing a superficial sum of the two. At the end, we would then have a more complete framework to solve physics-based differential equations with modern machine learning.

-
- [1] Z. Ren, S. Zhou, D. Liu, and Q. Liu, Physics-informed neural networks: A review of methodological evolution, theoretical foundations, and interdisciplinary frontiers toward next-generation scientific computing, *Applied Sciences* **15**, 10.3390/app15148092 (2025).
 - [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics informed deep learning (part I): data-driven solutions of nonlinear partial differential equations, *CoRR* **abs/1711.10561** (2017), 1711.10561.
 - [3] S. Cuomo, V. S. D. Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, Scientific machine learning through physics-informed neural networks: Where we are and what's next, *CoRR* **abs/2201.05624** (2022), 2201.05624.
 - [4] M. Hjorth-Jensen, Applied data analysis and machine learning (2025).
 - [5] H. E. H. Frederik Callin Østern, Viktor Bilstad, Regression analysis on runge function: gradient descent and resampling techniques, (2025).
 - [6] H. E. H. Frederik Callin Østern, Feed forward neural networks: Performance analysis on regression and classification, (2025).
 - [7] D. J. Griffiths and D. F. Schroeter, *Introduction to Quantum Mechanics*, 3rd ed. (Cambridge University Press, Cambridge, 2018).
 - [8] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics informed deep learning (part II): data-driven discovery of nonlinear partial differential equations, *CoRR* **abs/1711.10566** (2017), 1711.10566.
 - [9] Weights & Biases, Weights & biases (2025).